

Uninformed Search II: Action Costs and Uniform-Cost Search

Lecture 6

CS 4880

Kirk Duran

Today

- ▶ How Micromouse exposed limitations in a seemingly solved search formulation.
- ▶ Why richer action sets and physical motion constraints change the state space.
- ▶ How kinematics, velocity, and environmental dynamics affect feasible actions.
- ▶ Why unequal actions require an explicit action-cost model.
- ▶ How Dijkstra's algorithm / Uniform-Cost Search finds minimum-cost solutions.
- ▶ Where minimum-cost graph search appears as a baseline or subroutine in modern AI systems.

Key idea

Search quality depends not only on the search algorithm, but also on how we define the states, actions, transitions, and objective.

Part 1: Micromouse Revisited — When the Model Is Too Simple



Last Time: A Deliberately Simple Micromouse Model

Discrete search abstraction

- ▶ State: current maze cell
 - ▶ Actions: move to an adjacent open cell
 - ▶ Goal: reach a center cell
 - ▶ Objective: minimize the number of actions
-
- ▶ BFS could find a minimum-depth route once the maze was known.
 - ▶ Flood-fill style methods exploited the same grid structure efficiently.

Image Placeholder

`micromouse-simple-grid-model.png`

Key idea

The abstraction was useful because it removed physical details. Today we ask what happens when some of those details become important.

A Search Algorithm Can Only Use the Actions We Define

Early orthogonal action model

FORWARD
TURN LEFT 90°
TURN RIGHT 90°

- ▶ Paths are constructed from horizontal and vertical segments.
- ▶ A change in direction is represented as a 90-degree turn.
- ▶ Diagonal trajectories are not expressible in the model.

Image Placeholder

orthogonal-action-set-limits.png

Key idea

A solution cannot be discovered if the action representation makes that solution impossible to express.

Micromouse Risked Becoming a “Solved” Problem

- ▶ By the late 1980s, reliable maze discovery and shortest-path methods were well understood.
- ▶ The research community began to question whether the competition still presented substantial new technical challenges.
- ▶ The apparent problem had become: discover the maze, compute a shortest orthogonal route, then execute it faster.

But this conclusion depended on the formulation

If states and actions remain fixed, performance eventually becomes dominated by incremental implementation improvements.

Image Placeholder

`micromouse-apparent-stagnation.png`

Mitee Mouse III Changed the Action Space

- ▶ Mitee Mouse III demonstrated diagonal running rather than restricting every route to orthogonal grid segments.
- ▶ A 45-degree transition could enter or leave a diagonal corridor through the maze.
- ▶ The maze did not change; the **agent's action set** changed.

New possibilities

orthogonal: $\rightarrow \downarrow \rightarrow$

diagonal: $\rightarrow \searrow \swarrow \rightarrow$

Image Placeholder

mitee3-diagonal-running.png

Key idea

Expanding ACTIONS(s) can reveal qualitatively new solutions, not merely faster implementations of old ones.

Richer Actions Create a Richer Search Space

Orthogonal model

- ▶ forward
- ▶ 90-degree left
- ▶ 90-degree right

Richer motion primitives

- ▶ straight motion
- ▶ 90-degree smooth turns
- ▶ enter diagonal at 45 degrees
- ▶ diagonal travel
- ▶ leave diagonal at 45 degrees
- ▶ larger composite turns

- ▶ More actions increase the branching factor.
- ▶ More actions may also reduce solution length or execution time dramatically.
- ▶ Search complexity and representational power grow together.

Key idea

The action model is part of the intelligence: it determines what futures the agent is capable of considering.

Orientation Now Becomes Part of the State

Two mice can occupy the same maze cell but face different directions.

Too little information

$$s = (r, c)$$

Better state representation

$$s = (r, c, \theta)$$

- ▶ r, c describe location.
- ▶ θ describes orientation or heading.
- ▶ Available next actions depend on all three.

Image Placeholder

orientation-changes-actions.png

Key idea

A state must contain enough information for the transition model to predict the consequences of future actions.

Part 2: Motion Constraints — From Grid Search to Physical Planning



A Car Cannot Simply “Move Left”

A grid model might allow an action such as:

Abstract action

LEFT

A real car cannot translate sideways. Steering changes the vehicle's heading while it moves forward, producing an arc.

- ▶ Position changes continuously.
- ▶ Heading changes continuously.
- ▶ The vehicle has a nonzero turning radius.

Image Placeholder

car-left-action-is-arc.png

Key idea

Path planning must respect the motions the agent can physically execute, not merely geometric adjacency.

Kinematic Constraints Restrict Reachable States

Definition

Kinematics describes the geometry of motion—how position and orientation evolve—without modeling the forces that cause the motion.

A simple vehicle state may include:

$$s = (x, y, \theta)$$

where:

- ▶ x, y are position;
- ▶ θ is heading.

Steering limits may impose a minimum turning radius.

Image Placeholder

`kinematic-bicycle-model.png`

A Minimal Kinematic Bicycle Model

A common simplified model is:

$$\dot{x} = v \cos \theta$$

$$\dot{y} = v \sin \theta$$

$$\dot{\theta} = \frac{v}{L} \tan \delta$$

where:

- ▶ v is forward velocity,
- ▶ L is wheelbase,
- ▶ δ is steering angle.

Interpretation

- ▶ Steering does not instantaneously rotate the car.
- ▶ Turning rate depends on both speed and steering angle.
- ▶ Some geometrically short paths are not kinematically feasible.

Key idea

The transition model must encode the system's motion constraints; otherwise search can return physically impossible plans.

Velocity Can Change Which Actions Are Feasible

Consider two cars at the same position and heading:

Vehicle	Velocity
A	5 mph
B	60 mph

A maneuver that is feasible for A may be impossible or unsafe for B.

Richer state

$$s = (x, y, \theta, v)$$

Image Placeholder

velocity-dependent-stopping.png

Key idea

If future actions depend on velocity, velocity belongs in the state representation.

Stopping Distance Grows Rapidly with Speed

Under an idealized constant-deceleration model:

$$d_{\text{stop}} \approx \frac{v^2}{2a}$$

Low speed

At 5 mph, the kinetic energy and required braking distance are comparatively small.

High speed

At 60 mph, the same braking capability cannot stop the vehicle within the same short distance.

Key idea

Search over physical systems may require state-dependent actions: `ACTIONS(s)` changes as speed and configuration change.

Beyond Kinematics: Dynamics and External Conditions

Definition

Dynamics concerns how forces, mass, inertia, friction, and actuation determine motion.

The same steering command can produce different outcomes under different conditions:

- ▶ dry pavement,
- ▶ wet pavement,
- ▶ ice,
- ▶ loose gravel.

A simple friction-limited model gives roughly:

$$a_{\max} \approx \mu g$$

Image Placeholder

road-friction-changes-trajectory.png

The Transition Model Depends on the Physical World

Previously:

$$\text{RESULT}(s, a) = s'$$

looked almost like a table lookup.

For a physical agent, the result may depend on:

- ▶ current position and orientation;
- ▶ current velocity;
- ▶ steering and acceleration limits;
- ▶ friction and traction;
- ▶ environmental conditions.

Key idea

Problem formulation is a modeling decision: include enough physical detail to predict useful consequences without making planning computationally intractable.

Micromouse Faced the Same Physical Limits

As Micromouse speeds increased:

- ▶ turning required greater lateral acceleration;
- ▶ tire traction became a limiting factor;
- ▶ wheel cleanliness and surface friction mattered;
- ▶ later designs introduced suction fans to increase normal force and cornering capability.

The software and hardware co-evolved

Better planning was useful only when the robot could physically execute the resulting trajectory.

Image Placeholder

`micromouse-traction-and-fan.png`

Part 3: From Feasibility to Preference — Action Costs



Feasible Does Not Mean Equally Desirable

Suppose two routes both reach the goal.

Route A

- ▶ 5 actions
- ▶ many sharp turns
- ▶ repeated deceleration
- ▶ execution time: 8.2 s

Route B

- ▶ 7 actions
- ▶ mostly straight motion
- ▶ higher sustained speed
- ▶ execution time: 6.4 s

Key idea

Minimizing the number of actions is only appropriate when all actions are effectively equivalent.

Shortest Path Is Not Necessarily Fastest Path

The Micromouse competition provides a concrete example:

- ▶ a geometrically longer route can contain fewer or gentler turns;
- ▶ fewer costly turns can permit a higher sustained velocity;
- ▶ therefore a longer route can have lower execution time.

Objective matters

minimum distance $\neq$ minimum time

Image Placeholder

`micromouse-shortest-vs-fastest.png`

Action Cost

Definition

The **action cost** (or **step cost**) assigns a numerical cost to performing action a in state s and reaching state s' .

$$c(s, a, s')$$

Action cost can represent different objectives:

- ▶ execution time;
- ▶ physical distance;
- ▶ energy consumption;
- ▶ monetary cost;
- ▶ risk or exposure;
- ▶ a designed combination of several factors.

Key idea

The cost function defines what the search algorithm means by a “better” solution.

Path Cost Accumulates Action Costs

For a path

$$s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} s_2 \cdots \xrightarrow{a_n} s_n,$$

define cumulative path cost:

$$g(n) = \sum_{i=1}^n c(s_{i-1}, a_i, s_i).$$

Lecture 5 objective

Minimize solution depth:

$$d(n)$$

Lecture 6 objective

Minimize cumulative path cost:

$$g(n)$$

Depth and Cost Can Disagree

Consider two solutions:

Shallow solution

$S \rightarrow A \rightarrow G$
depth = 2, cost = 16

Deeper solution

$S \rightarrow B \rightarrow C \rightarrow G$
depth = 3, cost = 6

Image Placeholder

depth-versus-path-cost.png

Key idea

Once edge costs differ, the shallowest goal may not be the best goal.

Why Breadth-First Search Is No Longer Enough

BFS prioritizes:

smallest depth

but weighted search requires:

smallest accumulated cost

Frontier node	Depth	Path cost g
A	2	3
B	2	50
C	3	4

BFS sees A and B as equally shallow. A cost-aware search should prefer A, then possibly C, before B.

Key idea

We need a frontier policy ordered by cumulative path cost rather than depth.

Part 4: Dijkstra's Algorithm and Uniform-Cost Search



Edsger W. Dijkstra

- ▶ Dijkstra approached computing as a mathematical discipline centered on precise algorithms and proofs.
- ▶ In 1959, he published *A Note on Two Problems in Connexion with Graphs*.
- ▶ One of the algorithms in that paper computes shortest paths in a graph with non-negative edge weights.

Central idea

Repeatedly select the reachable unsettled vertex with the smallest known cumulative distance.

Image Placeholder

dijkstra-1959-paper.png

Dijkstra vs. Uniform-Cost Search

Dijkstra's algorithm

Typically presented as a single-source shortest-path algorithm that computes minimum costs from the source to reachable vertices.

Both use the same ordering principle:

$$\text{priority}(n) = g(n).$$

Uniform-Cost Search (UCS)

The AI search formulation: repeatedly expand the frontier node with minimum path cost and stop when a goal is removed for expansion.

Key idea

UCS is the goal-directed search form of Dijkstra's minimum-cost principle.

Frontier Policy: Always Expand the Cheapest Path So Far

Uniform-Cost rule

remove the frontier node with minimum $g(n)$

BFS

FIFO queue

priority = depth

DFS

LIFO stack

priority = depth-first order

UCS / Dijkstra

min-priority queue

priority = $g(n)$

Key idea

The algorithm remains uninformed: $g(n)$ measures only the cost already incurred, not how close n appears to the goal.

Worked Example: Weighted Search Graph

We will use the same graph for the next several slides.

Relevant edge costs

- ▶ $S \rightarrow A : 2$
- ▶ $S \rightarrow B : 5$
- ▶ $A \rightarrow C : 2$
- ▶ $A \rightarrow D : 5$
- ▶ $B \rightarrow D : 1$
- ▶ $C \rightarrow D : 1$
- ▶ $C \rightarrow G : 7$
- ▶ $D \rightarrow G : 2$

Image Placeholder

dijkstra-example-graph.png

Dijkstra Step 0: Initialize the Frontier

Initial values:

$$g(S) = 0$$

$$g(v) = \infty \quad \text{for all other vertices}$$

Frontier:

Priority queue

$(S, 0)$

Parent:

$$\text{parent}[S] = \text{None}$$

Image Placeholder

dijkstra-step0-initialize.png

Dijkstra Step 1: Expand S

Remove the minimum-cost frontier node:

$$S : 0$$

Relax its outgoing edges:

$$g(A) = 0 + 2 = 2$$

$$g(B) = 0 + 5 = 5$$

Frontier
$A : 2$
$B : 5$

Image Placeholder

dijkstra-step1-expand-s.png

Dijkstra Step 2: Expand A

The cheapest frontier node is:

$$A : 2$$

Relax successors:

$$g(C) = 2 + 2 = 4$$

$$g(D) = 2 + 5 = 7$$

New frontier:

Priority queue ordered by g

$$C : 4, \quad B : 5, \quad D : 7$$

Key idea

Search order is determined by cumulative cost, not by how visually close a node is to the goal.

Dijkstra Step 3: Relaxation Improves a Known Path

Expand:

$$C : 4$$

From C :

$$\text{candidate}(D) = 4 + 1 = 5$$

but our current best value is:

$$g(D) = 7.$$

Since $5 < 7$:

$$g(D) \leftarrow 5, \quad \text{parent}[D] \leftarrow C.$$

Key idea

Relaxation replaces a known route when a newly discovered route reaches the same state more cheaply.

Dijkstra Step 4: Reach the Goal at Minimum Cost

After the remaining relaxations:

- ▶ B has cost 5;
- ▶ D has cost 5;
- ▶ expanding D produces G with cost $5 + 2 = 7$;
- ▶ the earlier route $C \rightarrow G$ would cost $4 + 7 = 11$.

When $G : 7$ becomes the minimum item removed from the priority queue, the solution is final.

Minimum-cost path

$$S \rightarrow A \rightarrow C \rightarrow D \rightarrow G$$

$$\text{total cost} = 2 + 2 + 1 + 2 = 7$$

Key idea

For non-negative edge costs, the goal is safe to return when it is removed as the minimum-cost frontier node—not merely when it is first generated.

Uniform-Cost Search Pseudocode

Algorithm

```
frontier ← PriorityQueue()
frontier.insert(initial_state, priority=0)
cost_so_far[initial_state] ← 0
came_from[initial_state] ← None

while frontier is not empty:
    state ← frontier.remove_min()
    if goal_test(state):
        return reconstruct_path(came_from, state)
    for action in ACTIONS(state):
        child ← RESULT(state, action)
        new_cost ← cost_so_far[state] + ACTION_COST(state, action, child)
        if child is new or new_cost < cost_so_far[child]:
            cost_so_far[child] ← new_cost
            came_from[child] ← state
            frontier.insert(child, priority=new_cost)
return failure
```

Why Is the Cheapest Frontier Node Safe to Finalize?

Suppose n is the frontier node with minimum cost $g(n)$.

Could an undiscovered path later reach n for less?

1. Any alternate path must first pass through some not-yet-expanded frontier state m .
2. By construction, $g(m) \geq g(n)$.
3. All remaining edge costs are non-negative.
4. Continuing from m cannot reduce the accumulated cost below $g(m)$.

Therefore no later path can improve n once it is removed as the cheapest frontier state.

Key idea

Non-negative edge costs are what make Dijkstra's greedy finalization step correct.

Properties of Dijkstra / Uniform-Cost Search

Property	Result
Complete	Yes on a finite graph with non-negative edge costs
Optimal	Yes; returns a minimum-cost path
Frontier	Min-priority queue ordered by $g(n)$
Time	$O((V + E) \log V)$ with adjacency list + binary heap
Auxiliary search space	$O(V)$ for distances, parents, and frontier metadata

Restriction

Dijkstra's finalization rule is not valid in the presence of negative-cost edges.

What Dijkstra Pays for Its Guarantee

Strengths

- ▶ complete under standard assumptions;
- ▶ minimum-cost guarantee;
- ▶ domain-independent;
- ▶ handles arbitrary non-negative edge costs.

Costs

- ▶ potentially large frontier;
- ▶ stores best known costs and parents;
- ▶ can expand many states irrelevant to the goal;
- ▶ uses no estimate of remaining distance.

Key idea

Dijkstra is cost-aware, but it is still uninformed about where the goal lies.

Part 5: Changing the Cost Function Changes the Solution



Same Puzzle, Different Objective

Consider the sliding-tile puzzle from Lecture 5.

Uniform action model

Every legal tile move costs 1.

Objective: minimize number of moves.

Weighted action model

Moving tile k costs k .

Objective: minimize sum of moved tile values.

Image Placeholder

weighted-sliding-tile-puzzle.png

Key idea

States and legal transitions can remain identical while the preferred solution changes because the objective changed.

Micromouse: Cost Can Approximate Execution Time

A time-oriented action model might assign different costs to different motion primitives.

Motion primitive	Example modeled cost
long straight segment	low cost per unit distance
gentle curve	moderate cost
sharp 90-degree turn	higher cost
enter / exit diagonal	transition cost
braking and re-acceleration	additional time cost

Key idea

The search objective should reflect the performance metric we actually care about—for racing, that may be time rather than geometric distance.

State, Actions, Transitions, and Costs Work Together

Simple maze model

- ▶ State: cell
- ▶ Actions: orthogonal moves
- ▶ Transition: adjacent cell
- ▶ Cost: 1 per move

Richer physical model

- ▶ State: position, heading, velocity
- ▶ Actions: motion primitives
- ▶ Transition: kinematic/dynamic model
- ▶ Cost: predicted execution time

Key idea

Better planning often comes from improving the problem formulation, not merely replacing the search algorithm.

Part 6: Where Minimum-Cost Search Appears in AI



Robotics and Autonomous Navigation

Weighted graph search is a natural baseline when a robot must trade among:

- ▶ travel distance;
- ▶ estimated execution time;
- ▶ energy use;
- ▶ terrain difficulty;
- ▶ clearance or collision risk.

Typical abstraction

roadmap / occupancy grid → weighted graph → minimum-cost route

Image Placeholder

robot-weighted-navigation.png

Autonomous Driving: Global Route Cost

A driving route may be evaluated using quantities such as:

- ▶ travel time;
- ▶ road length;
- ▶ speed limits;
- ▶ turn penalties;
- ▶ road class;
- ▶ predicted congestion or risk.

Important distinction

Real autonomous-driving stacks use much richer models than plain Dijkstra. Dijkstra provides the foundational minimum-cost graph-search principle over a discretized route network.

Key idea

Weighted graph search often solves the global route problem; local motion planning must additionally respect vehicle dynamics and nearby obstacles.

Symbolic Planning with Nonuniform Actions

An AI planner may reason over actions such as:

drive fly charge purchase wait

Different actions may incur different:

- ▶ durations;
- ▶ monetary costs;
- ▶ resource consumption;
- ▶ risk penalties.

The state-space graph can therefore be searched for a minimum-cost plan rather than a minimum-number-of-actions plan.

Key idea

Uniform-cost search is a general planning primitive whenever action sequences have additive non-negative costs.

Game and Simulation AI

Navigation graphs and navigation meshes often encode nonuniform traversal preferences.

Region	Example cost
open floor	1
mud / difficult terrain	3
exposed danger zone	10

An AI-controlled character may choose a geometrically longer route because it has lower modeled total cost.

Key idea

“Shortest” is meaningful only after the system designer specifies the metric being minimized.

Knowledge Graphs: Useful Subroutine, Not the Whole System

Knowledge graphs represent:

- ▶ entities as vertices;
- ▶ relationships as edges;
- ▶ confidence, distance, or relation penalties as possible weights.

Weighted shortest-path computations can support tasks such as:

- ▶ finding low-cost relation chains;
- ▶ measuring graph proximity;
- ▶ generating candidate explanation paths;
- ▶ ranking graph-connected candidates.

Modern AI context

Entity linking and knowledge-graph reasoning are usually not “just Dijkstra.” Learned representations, retrieval models, language models, and graph neural methods often provide the dominant signal. Shortest-path search is better viewed as a reusable graph primitive inside a larger system.

Search Appears Inside Larger AI Architectures

perception / retrieval / learned model $\longrightarrow$ weighted graph or planning problem $\longrightarrow$ search $\longrightarrow$
action or explanation

- ▶ AI systems frequently combine learned components with classical algorithms.
- ▶ A learned model may estimate costs or construct the graph.
- ▶ A search algorithm may then enforce global consistency or find a minimum-cost sequence.

Key idea

Classical search and modern machine learning are complementary computational tools, not mutually exclusive definitions of AI.

Part 7: Why Dijkstra Is Still Uninformed

Cost-Aware Does Not Mean Goal-Informed

Dijkstra / UCS knows:

$$g(n) = \text{cost already paid to reach } n.$$

It does **not** know:

- ▶ how far n is from the goal;
- ▶ whether n points in a promising direction;
- ▶ whether a neighboring region is likely to contain a solution.

Decision rule

“Which discovered state has been cheapest to reach so far?”

Key idea

Dijkstra uses path cost, but no estimate of remaining cost. It is therefore still an uninformed search strategy.

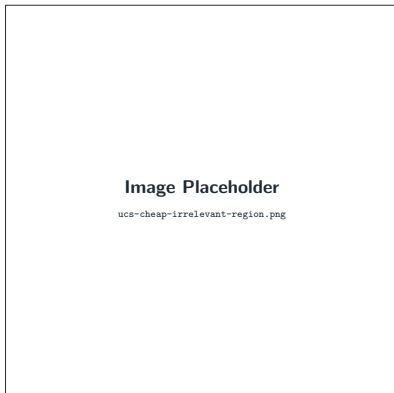
The Cost Contour Can Expand in the Wrong Direction

Suppose the goal lies to the east, but a large region to the south contains many very cheap edges.

Dijkstra may expand much of that region first because:

$$g(\text{southern states}) < g(\text{goal route}).$$

Nothing in $g(n)$ says which direction leads toward the goal.



Key idea

Optimality can require substantial exploration when the search has no estimate of remaining distance.

Lecture 5 vs. Lecture 6

	Lecture 5	Lecture 6
Action model	effectively equal steps	variable costs
Objective	minimum depth	minimum path cost
Main frontier	FIFO / stack / depth limit	priority queue
Key strategies	BFS, DFS, IDDFS	Dijkstra / UCS
Goal knowledge	none	none

Key idea

We changed what “best” means, but we still have no domain-specific estimate of which frontier state is closest to success.

Search Quality Depends on the Model

The Micromouse story gives us a complete modeling ladder:

1. Change the **state**: orientation and velocity become relevant.
2. Change the **actions**: diagonal and smooth motion becomes possible.
3. Change the **transition model**: physical motion constraints matter.
4. Change the **cost function**: fastest can replace shortest.
5. Change the **frontier policy**: minimum-cost search becomes necessary.

Key idea

An AI agent searches the world described by its model. Improving the model can change both which solutions exist and which solution counts as best.

Exit Ticket: Formulate the Right Search Problem

In class

An autonomous delivery robot can either cross an open courtyard or travel around it on a covered walkway. Rain makes the courtyard slippery and slows the robot.

Specify:

1. what additional state or environment information may matter;
2. how `ACTIONS(s)` could change;
3. what the action cost should represent;
4. why BFS may return a different route from Uniform-Cost Search.

Key idea

Outcome: you can explain when a uniform-step search model is insufficient and formulate a minimum-cost search problem that reflects physical or task-specific constraints.

Next Time: Heuristic Search

Dijkstra asks:

Known cost

How much has this path cost so far? $g(n)$

Suppose the agent could also estimate:

Remaining cost

How much more might it cost to reach the goal? $h(n)$

Then search can use information about the goal rather than expanding solely by accumulated cost.

Key idea

Next: Greedy Best-First Search, heuristic functions, and A^* .

Image Placeholder Index

- ▶ micromouse-simple-grid-model.png
- ▶ orthogonal-action-set-limits.png
- ▶ micromouse-apparent-stagnation.png
- ▶ mitee3-diagonal-running.png
- ▶ orientation-changes-actions.png
- ▶ car-left-action-is-arc.png
- ▶ kinematic-bicycle-model.png
- ▶ velocity-dependent-stopping.png
- ▶ road-friction-changes-trajectory.png
- ▶ micromouse-traction-and-fan.png
- ▶ micromouse-shortest-vs-fastest.png
- ▶ depth-versus-path-cost.png
- ▶ dijkstra-1959-paper.png
- ▶ dijkstra-example-graph.png
- ▶ dijkstra-step0-initialize.png
- ▶ dijkstra-step1-expand-s.png
- ▶ weighted-sliding-tile-puzzle.png
- ▶ robot-weighted-navigation.png
- ▶ ucs-cheap-irrelevant-region.png

Example

Save each image under `../assets/lecture-06/`. The labeled placeholder will be replaced automatically when its filename matches.